

Holistically-Nested Edge Detection (HED) Module for OpenCV contrib

A Learning-Based Alternative to Canny Edge Detection

SASANK

June 2026

Abstract

This report documents a new OpenCV `contrib` module, `hed`, that adds *Holistically-Nested Edge Detection* (HED) as an alternative to the classical Canny edge detector. Canny operates purely on local intensity gradients, so it reacts to texture, noise, and lighting changes that do not correspond to real object boundaries. HED instead uses a pretrained VGG-based convolutional network that was trained on human-annotated boundaries, producing semantically meaningful edge maps. The module wraps the network through OpenCV's `dnn` engine, exposes a small `HEDDetector` API to both C++ and Python, and ships with a demo, unit tests, and build integration.

Contents

1	Motivation: Why Replace Canny?	2
2	Summary of Improvements	2
3	Module Layout	3
4	Public API	3
5	Implementation	4
6	Build Integration	5
7	Usage	5
7.1	C++	5
7.2	Python	5
7.3	Demo: HED vs. Canny	5
8	Testing	6
9	Conclusion	7

1 Motivation: Why Replace Canny?

OpenCV’s standard edge detector, `cv::Canny`, is a hand-crafted pipeline: Gaussian smoothing, Sobel gradients, non-maximum suppression, and hysteresis thresholding. It is fast and parameter-light, but it has well-known limitations:

- **Local-only decisions.** Each edge pixel is decided from a tiny neighbourhood. There is no notion of an *object*; a strong texture edge and a true object boundary look identical.
- **Threshold sensitivity.** The two hysteresis thresholds must be tuned per image. Wrong values give either broken contours or a storm of spurious edges.
- **Noise and texture clutter.** Grass, fabric, foliage, and compression artifacts all produce gradients and therefore false edges.
- **No semantics.** Canny cannot tell that the outline of a person is more important than the wrinkles on their shirt.

HED addresses these by *learning* what an edge is from data. It is a fully convolutional network with deep supervision at multiple scales whose side-outputs are fused into a single boundary probability map. Because it was trained on the BSDS500 dataset of human-drawn boundaries, it produces clean, object-level edges with no per-image threshold tuning.

2 Summary of Improvements

Aspect	Canny (classic)	HED (this module)
Decision basis	Local intensity gradient	Learned multi-scale features
Semantics	None	Object-level boundaries
Parameters	2 thresholds, tuned per image	None at inference
Noise / texture	Sensitive	Robust
Output	Binary edge mask	Continuous probability map $[0, 1]$
Scale awareness	Single scale	Multi-scale side outputs fused
Training data	Not applicable	Human-annotated boundaries (BSDS500)

Table 1: Canny vs. the HED module at a glance.

Key benefits in brief:

1. **Semantic edges** — detects real object contours, not raw contrast.
2. **No tuning** — no thresholds to set; the same model works across images.
3. **Noise robustness** — texture and clutter are largely suppressed.
4. **Soft confidence output** — a probability map (CV_32F , $[0, 1]$) that downstream code can threshold flexibly.
5. **Seamless integration** — runs on OpenCV’s existing `dnn` backend; no extra dependency, and it is exposed to Python automatically.

3 Module Layout

The module follows the standard OpenCV contrib structure:

```
modules/hed/
  CMakeLists.txt
  README.md
  include/opencv2/hed.hpp      # public API
  src/hed.cpp                  # implementation (dnn-backed)
  samples/hed_demo.py          # HED vs Canny demo
  test/test_hed.cpp            # unit tests
```

4 Public API

The public header declares a single abstract class, `HEDDetector`, with a factory `create()` and one `detectEdges()` call. The `CV_EXPORTS_W` / `CV_WRAP` markers generate the Python bindings.

```
1 #pragma once
2 #include "opencv2/core.hpp"
3 #include <string>
4
5 namespace cv {
6 namespace hed {
7
8 /** @defgroup hed Holistically-nested Edge Detection
9 This module implements HED edge detection as described in:
10
11 Xie, S., & Tu, Z. (2015). Holistically-nested edge detection.
12 Proceedings of the IEEE international conference on computer vision.
13 https://arxiv.org/abs/1504.06375
14 */
15
16 //! @addtogroup hed
17 //! @{
18
19 /** @brief HED edge detector using a pretrained Caffe model.
20
21 Unlike classical detectors such as Canny, HED uses a VGG-based
22 convolutional network trained on human-annotated boundaries to
23 produce semantically meaningful edge maps.
24
25 @note Requires hed_pretrained_bsds.caffemodel and deploy.prototxt.
26 Download from: https://vcl.ucsd.edu/hed/
27 */
28 class CV_EXPORTS_W HEDDetector {
29 public:
30     /** @brief Creates an HEDDetector instance.
31     @param modelPath path to hed_pretrained_bsds.caffemodel
32     @param protoPath path to deploy.prototxt
33     */
34     CV_WRAP static cv::Ptr<HEDDetector> create(
35         const std::string& modelPath,
36         const std::string& protoPath
37     );
38
39     /** @brief Detects edges in an image.
40     @param image input BGR image (CV_8UC3)
```

```

41     @return edge map as CV_32F with values in [0, 1]
42     */
43     CV_WRAP virtual cv::Mat detectEdges(cv::InputArray image) = 0;
44
45     /** @brief Destructor. */
46     virtual ~HEDDetector() {}
47 };
48
49 //! @}
50
51 } // namespace hed
52 } // namespace cv

```

Listing 1: include/opencv2/hed.hpp

5 Implementation

The implementation loads the Caffe model through `cv::dnn::readNet` and runs a forward pass. The input is converted to a blob with the network's mean-subtraction values (the BGR means HED was trained with). The fused sigmoid output ("sigmoid-fuse") of shape $[1, 1, H, W]$ is squeezed to an $H \times W$ single-channel float map and returned as an owned copy.

```

1  #include "opencv2/hed.hpp"
2  #include "opencv2/dnn.hpp"
3  #include "opencv2/imgproc.hpp"
4
5  namespace cv {
6  namespace hed {
7
8  class HEDDetectorImpl : public HEDDetector {
9  public:
10     dnn::Net net;
11
12     HEDDetectorImpl(const std::string& modelPath, const std::string& protoPath) {
13         net = dnn::readNet(modelPath, protoPath);
14     }
15
16     cv::Mat detectEdges(cv::InputArray _image) override {
17         cv::Mat image = _image.getMat();
18         cv::Mat blob = dnn::blobFromImage(
19             image, 1.0,
20             cv::Size(image.cols, image.rows),
21             cv::Scalar(104.00698793, 116.66876762, 122.67891434),
22             false, false
23         );
24         net.setInput(blob);
25         cv::Mat result = net.forward("sigmoid-fuse");
26
27         // result shape is [1,1,H,W] -- squeeze to [H,W]
28         cv::Mat edge(result.size[2], result.size[3], CV_32F, result.ptr<float>());
29         cv::Mat output;
30         edge.copyTo(output);
31         return output;
32     }
33 };
34
35 cv::Ptr<HEDDetector> HEDDetector::create(

```

```

36     const std::string& modelPath,
37     const std::string& protoPath)
38 {
39     return cv::makePtr<HEDDetectorImpl>(modelPath, protoPath);
40 }
41
42 } // hed
43 } // cv

```

Listing 2: src/hed.cpp

Implementation notes.

- **Mean subtraction** uses the exact training means (104.007, 116.669, 122.679), so the input statistics match what the network expects.
- **Full-resolution inference:** the blob is sized to the input image, so the edge map comes back at the original resolution.
- **Owned output:** `edge.copyTo(output)` copies out of the network’s internal buffer so the returned `Mat` stays valid after the next forward pass.

6 Build Integration

The module declares its dependencies and enables Python wrapping in two lines. The dependency on `opencv_dnn` is what lets HED run without any external framework.

```

set(the_module opencv_hed)
ocv_define_module(hed opencv_core opencv_imgproc opencv_dnn WRAP python)

```

Listing 3: CMakeLists.txt

7 Usage

7.1 C++

```

auto det = cv::hed::HEDDetector::create("model.caffemodel", "deploy.prototxt");
cv::Mat edges = det->detectEdges(image); // CV_32F, values 0..1

```

7.2 Python

The bindings are generated automatically from the `CV_WRAP` markers.

```

detector = cv2.hed.HEDDetector.create("model.caffemodel", "deploy.prototxt")
edges = detector.detectEdges(image) # float32, values 0..1

```

7.3 Demo: HED vs. Canny

The shipped sample runs both detectors on the same image for a side-by-side comparison, making the qualitative improvement directly visible.

```

1 import cv2
2 import sys
3 import argparse
4
5 parser = argparse.ArgumentParser()
6 parser.add_argument('--image', required=True)
7 parser.add_argument('--model', required=True)
8 parser.add_argument('--proto', required=True)
9 args = parser.parse_args()
10
11 image = cv2.imread(args.image)
12 if image is None:
13     print(f"Cannot load image: {args.image}")
14     sys.exit(1)
15
16 detector = cv2.hed.HEDDetector.create(args.model, args.proto)
17 edges = detector.detectEdges(image)
18 edges_u8 = (edges * 255).astype('uint8')
19
20 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
21 canny = cv2.Canny(gray, 100, 200)
22
23 cv2.imshow("Original", image)
24 cv2.imshow("Canny (classic)", canny)
25 cv2.imshow("HED (yours)", edges_u8)
26 cv2.imwrite("hed_result.png", edges_u8)
27 print("Saved hed_result.png")
28 cv2.waitKey(0)

```

Listing 4: samples/hed_demo.py

8 Testing

Two unit tests cover model loading and output contract. They skip gracefully when the model files are not present in the test data path, so the suite stays green on machines without the weights.

```

1 #include "test_precomp.hpp"
2
3 namespace opencv_test { namespace {
4
5 TEST(HED, LoadModel)
6 {
7     std::string model = cvtest::findDataFile("hed_pretrained_bsds.caffemodel",
8     false);
9     std::string proto = cvtest::findDataFile("deploy.prototxt", false);
10    if (model.empty() || proto.empty())
11        throw SkipTestException("HED model files not found");
12
13    auto detector = cv::hed::HEDDetector::create(model, proto);
14    ASSERT_FALSE(detector.empty());
15 }
16
17 TEST(HED, OutputShape)
18 {
19     std::string model = cvtest::findDataFile("hed_pretrained_bsds.caffemodel",
20     false);

```

```

19     std::string proto = cvtest::findDataFile("deploy.prototxt", false);
20     if (model.empty() || proto.empty())
21         throw SkipTestException("HED model files not found");
22
23     cv::Mat img = cv::Mat::zeros(100, 100, CV_8UC3);
24     auto detector = cv::hed::HEDDetector::create(model, proto);
25     cv::Mat edges = detector->detectEdges(img);
26
27     EXPECT_EQ(edges.rows, 100);
28     EXPECT_EQ(edges.cols, 100);
29     EXPECT_EQ(edges.type(), CV_32F);
30 }
31
32 }} // namespace
33
34 CV_TEST_MAIN(".")

```

Listing 5: test/test_hed.cpp

9 Conclusion

The `hed` module brings a modern, learning-based edge detector into the OpenCV ecosystem as a drop-in alternative to Canny. By replacing hand-tuned gradient thresholds with a network trained on human boundary annotations, it yields edge maps that are cleaner, semantically meaningful, parameter-free at inference, and robust to texture and noise. It integrates through the existing `dnn` backend, ships C++ and auto-generated Python APIs, and includes a comparison demo and unit tests — making it ready for use in segmentation, contour extraction, and other downstream pipelines where Canny’s local heuristics fall short.

Reference. Xie, S., & Tu, Z. (2015). *Holistically-Nested Edge Detection*. ICCV. <https://arxiv.org/abs/1504.06375>